

# 13. TEST CLOSURE PHASE

The Test Closure Phase is the final stage of the Software Testing Life Cycle (STLC). This phase is performed after all planned testing activities have been completed. The primary purpose of test closure is to evaluate testing results, document key findings, measure testing effectiveness, identify lessons learned, and formally conclude the testing process.

In this project, the Test Closure Phase was conducted after the successful execution of Login Testing, SQL Injection Testing, Cross-Site Scripting (XSS) Testing, OWASP ZAP Security Scans, and Report Generation activities. All execution results, vulnerabilities, reports, and observations were reviewed and documented to ensure project objectives were achieved.

---

## 13.1 Execution Summary

The execution summary provides an overall view of the testing activities performed during the project.

The automation framework successfully executed functional and security test cases against the Damn Vulnerable Web Application (DVWA). The execution process included authentication testing, SQL Injection testing, Cross-Site Scripting (XSS) testing, OWASP ZAP vulnerability scanning, and report generation.

Throughout the testing process, both automated scripts and security scanning tools were used to validate application behavior and identify vulnerabilities.

The following activities were completed successfully:

### Functional Testing Activities

- Login functionality validation
- Authentication verification
- Navigation verification
- Session establishment validation

### Security Testing Activities

- SQL Injection Testing
- Cross-Site Scripting (XSS) Testing
- Input Validation Testing
- Vulnerability Assessment

### Security Scanning Activities

- OWASP ZAP Spider Scan
- OWASP ZAP Active Scan

- Vulnerability Detection
- Risk Classification

**Reporting Activities**

- TestNG Report Generation
- Extent Report Generation
- OWASP ZAP Security Report Generation

The testing process successfully identified several vulnerabilities intentionally present within DVWA, demonstrating the effectiveness of the security testing framework.

---

**Execution Status Summary**

| Testing Activity      | Status    |
|-----------------------|-----------|
| Login Testing         | Completed |
| SQL Injection Testing | Completed |
| XSS Testing           | Completed |
| OWASP ZAP Spider Scan | Completed |
| OWASP ZAP Active Scan | Completed |
| TestNG Reporting      | Completed |
| Extent Reporting      | Completed |
| Security Reporting    | Completed |

Overall Project Status:

**Successfully Completed**

---

**13.2 Test Metrics**

Test Metrics are quantitative measurements used to evaluate the effectiveness and quality of the testing process.

The following metrics were collected during project execution.

# Test Case Metrics

## Total Test Cases Designed

| Module               | Count |
|----------------------|-------|
| Login Module         | 4     |
| SQL Injection Module | 4     |
| XSS Module           | 4     |
| OWASP ZAP Module     | 4     |
| Reporting Module     | 4     |
| Total                | 20    |

---

## Test Execution Results

| Status         | Count |
|----------------|-------|
| Passed         | 15    |
| Failed         | 5     |
| Skipped        | 0     |
| Total Executed | 20    |

---

## Pass Percentage

Formula:

$$\text{Pass Percentage} = (\text{Passed Test Cases} / \text{Total Executed Test Cases}) \times 100$$

Calculation:

$$(15 / 20) \times 100 = 75\%$$

Pass Rate:

**75%**

---

## Fail Percentage

Formula:

$$\text{Fail Percentage} = (\text{Failed Test Cases} / \text{Total Executed Test Cases}) \times 100$$

Calculation:

$$(5 / 20) \times 100 = 25\%$$

Fail Rate:

25%

---

## Vulnerability Metrics

The following vulnerabilities were identified during testing.

| Vulnerability Type         | Count |
|----------------------------|-------|
| SQL Injection              | 1     |
| Cross-Site Scripting (XSS) | 1     |
| Missing Security Headers   | 3     |
| Cookie Security Issues     | 2     |
| Information Disclosure     | 1     |
| Total Findings             | 8     |

---

## Severity Distribution

| Severity | Count |
|----------|-------|
| Critical | 2     |
| High     | 2     |
| Medium   | 3     |
| Low      | 1     |

---

## Automation Metrics

| Metric               | Value             |
|----------------------|-------------------|
| Framework Type       | Selenium + TestNG |
| Programming Language | Java              |
| Build Tool           | Maven             |
| Security Tool        | OWASP ZAP         |
| Reports Generated    | 3                 |
| Browser Used         | Chrome            |

---

## Report Metrics

Reports generated during execution:

- TestNG Report
- Extent Report
- OWASP ZAP Security Report

Total Reports Generated:

---

## 13.3 Lessons Learned

The Lessons Learned section captures important observations, experiences, and knowledge gained during project execution.

These lessons help improve future projects and testing activities.

---

### Understanding Security Testing

One of the most important learnings from this project was understanding the significance of security testing in modern web applications.

Security vulnerabilities can lead to:

- Unauthorized access
- Data theft
- Information disclosure
- Financial loss
- Reputation damage

Early identification of vulnerabilities significantly improves application security.

---

### Practical Experience with OWASP ZAP

The project provided hands-on experience with OWASP ZAP.

Key learnings included:

- Configuring ZAP API settings
- Executing Spider Scans
- Running Active Scans
- Interpreting vulnerability findings
- Generating security reports

This experience improved understanding of automated vulnerability assessment techniques.

---

### Automation Framework Development

The project improved knowledge of automation framework design using:

- Selenium WebDriver
- Java
- TestNG
- Maven

Key learnings included:

- Page Object Model implementation
  - TestNG framework usage
  - Data-driven testing
  - Report generation
  - Dependency management
- 

## **Importance of Environment Configuration**

Several execution challenges highlighted the importance of proper environment setup.

Examples included:

- ZAP connectivity issues
- Browser compatibility problems
- Dependency conflicts
- Database configuration errors

Proper environment validation significantly reduces execution failures.

---

## **Importance of Security Best Practices**

The project reinforced the need for secure coding practices.

Important practices include:

- Input validation
- Output encoding
- Parameterized queries
- Security headers
- Secure session management

Implementing these controls can prevent many common vulnerabilities.

---

## **Documentation and Reporting**

Comprehensive documentation proved valuable for:

- Tracking execution progress
- Recording findings
- Communicating risks
- Supporting future maintenance

Well-prepared reports improve project transparency and professionalism.

---

## 13.4 Closure Report

The Closure Report formally concludes all testing activities performed during the project.

The Web Application Security Testing Project was executed successfully using Selenium WebDriver, TestNG, Maven, and OWASP ZAP. The testing framework successfully automated functional and security testing activities against the DVWA application.

The project achieved its primary objectives, including:

- Development of an automated testing framework.
- Successful execution of login automation.
- Identification of SQL Injection vulnerabilities.
- Identification of Cross-Site Scripting vulnerabilities.
- Successful execution of OWASP ZAP Spider and Active Scans.
- Generation of TestNG, Extent, and Security Reports.
- Documentation of security findings and recommendations.

The security assessment identified multiple vulnerabilities intentionally present within DVWA. These findings validated the effectiveness of both automated testing techniques and OWASP ZAP scanning capabilities.

The project demonstrated how automation can be integrated with security testing tools to improve vulnerability detection, reduce manual effort, and support secure software development practices.

---

## Final Project Outcome

| Objective         | Status    |
|-------------------|-----------|
| Environment Setup | Completed |

|                       |           |
|-----------------------|-----------|
| Framework Development | Completed |
| Login Testing         | Completed |
| SQL Injection Testing | Completed |
| XSS Testing           | Completed |
| OWASP ZAP Integration | Completed |
| Security Scanning     | Completed |
| Report Generation     | Completed |
| Documentation         | Completed |

## Overall Project Status

### SUCCESSFULLY COMPLETED

---

The Test Closure Phase confirms that all planned testing activities were executed, project objectives were achieved, vulnerabilities were successfully identified, reports were generated, and the Web Application Security Testing Framework was completed successfully.